

05 · GraphQL: consultas básicas

GraphQL cambia el trato con el servidor:

- **Un solo endpoint:** `POST /api/graphql` (no una URL por recurso).
- **Tú describes la forma exacta** de la respuesta: pides los campos que quieres y los recibes anidados igual que en la query.
- En `tienda-virtual` es **solo lectura** (las escrituras van por REST) y usa la **misma autenticación dual** que el REST: `x-api-key` o `Authorization: Bearer`.

Detalle importante: en GraphQL un error de consulta llega con **HTTP 200** y un array `errors` en el cuerpo. Hay que revisarlo siempre.

```
import csv
import os

import requests

GQL = "http://localhost:3000/api/graphql"
HEADERS = {"x-api-key": "sk_demo_00000000000000000000000000000000"}
TIMEOUT = 30

def gql(query, variables=None):
    r = requests.post(
        GQL,
        json={"query": query, "variables": variables or {}},
        headers=HEADERS,
        timeout=TIMEOUT,
    )
    r.raise_for_status() # errores de transporte / auth (401)
    cuerpo = r.json()
    if "errors" in cuerpo: # errores de la propia query (llegan con HTTP 200)
        raise RuntimeError(cuerpo["errors"])
    return cuerpo["data"]
```

Una query plana: productos con su rating agregado

```
QUERY = """
{
  productos(page: 1, pageSize: 5) {
    items {
      id codigo nombre precio stock
      aggregateRating { ratingValue reviewCount }
    }
    pageInfo { page pageSize total totalPages }
  }
}
```

```
data = gql(QUERY)
print("pageInfo:", data["productos"]["pageInfo"])
for p in data["productos"]["items"]:
    ar = p["aggregateRating"]
    valor = f"{{ar['ratingValue']}} ({{ar['reviewCount']}})" if ar else "-"
    print(f"  {{p['id']:>2}}  {{p['nombre'][:38]:38s}} S/  {{p['precio']:>7}}  {{valor}}")
```

```
pageInfo: {'page': 1, 'pageSize': 5, 'total': 90, 'totalPages': 18}
[ 1] Apple iPhone 15 Pro Max 256GB      S/      5599   3.5 (4)
[ 2] Apple iPhone 15 128GB             S/      3799   3.3 (4)
[ 3] Samsung Galaxy S24 Ultra 512GB     S/      5299   3.3 (4)
[ 4] Samsung Galaxy A55 128GB          S/      1399   3 (4)
[ 5] Xiaomi Redmi Note 13 Pro 256GB     S/       999   2.8 (4)
```

Un campo que no existe: errors con HTTP 200

```
try:
    gql("{ productos { items { campoInexistente } } }")
except RuntimeError as e:
    print("GraphQL devolvió errores (y aun así HTTP 200):")
    for err in e.args[0]:
        print("  -", err["message"])
```

GraphQL devolvió errores (y aun así HTTP 200):
 - Cannot query field "campoInexistente" on type "Producto".

Introspección: preguntarle al servidor qué queries ofrece

GraphQL se describe a sí mismo. Con `__schema` obtenemos la lista de consultas disponibles y sus argumentos (útil para explorar una API nueva).

```
INTROSPECCION = """
{
  __schema {
    queryType {
      fields { name args { name } }
    }
  }
}
"""

for f in gql(INTROSPECCION)["__schema"]["queryType"]["fields"]:
    args = ", ".join(a["name"] for a in f["args"])
    print(f"  {{f['name']}}({{args}})")
```

```
productos(page, pageSize)
producto(id)
clientes(page, pageSize)
cliente(id)
ordenes(page, pageSize, fecha, clienteId)
orden(id)
testimonios(page, pageSize)
comentarios(tipo, productoId, ordenId, clienteId)
```

Paginación con variables y guardado en data/graphql_productos.csv

Las **variables** (`$page` , `$pageSize`) evitan concatenar strings en la query.

```
QUERY_PAGINA = """
query Catalogo($page: Int!, $pageSize: Int!) {
  productos(page: $page, pageSize: $pageSize) {
    items {
      id codigo nombre categoria subcategoria precio stock
      aggregateRating { ratingValue reviewCount }
    }
  }
  pageInfo { page totalPages total }
}
"""

def todas_las_paginas(page_size=50):
    items = []
    page = 1
    while True:
        d = gql(QUERY_PAGINA, {"page": page, "pageSize": page_size})["productos"]
        items.extend(d["items"])
        info = d["pageInfo"]
        print(f"  página {info['page']}>2}/{info['totalPages']} (total
{len(items)}/{info['total']})")
        if info["page"] >= info["totalPages"]:
            return items
        page += 1

productos = todas_las_paginas()

DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT = os.path.join(DATA_DIR, "graphql_productos.csv")
with open(OUTPUT, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=["id", "codigo", "nombre", "categoria", "subcategoria", "precio",
"stock", "rating_promedio", "total_resenas"],
    )
    writer.writeheader()
    for p in productos:
        ar = p["aggregateRating"] or {}
        writer.writerow({
            "id": p["id"], "codigo": p["codigo"], "nombre": p["nombre"],
            "categoria": p["categoria"], "subcategoria": p["subcategoria"],
            "precio": p["precio"], "stock": p["stock"],
            "rating_promedio": ar.get("ratingValue", ""),
            "total_resenas": ar.get("reviewCount", ""),
        })

print(f"\nGuardadas {len(productos)} filas en {OUTPUT}")
```

```
página 1/2 (total 50/90)
página 2/2 (total 90/90)
```

Guardadas 90 filas en /private/tmp/claude-501/Users/ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/graphql_productos.csv

En el siguiente notebook aprovechamos de verdad GraphQL: una sola query con datos **anidados** (órdenes -> cliente -> items -> producto) y autenticación por OAuth.